26PDPA0 - DATA MODELLING AND VISUALIZATION

Assignment 2a

MongoDB Exercises and NoSQL Data Models

Document model | Wide-column model | Graph model

Student name	_____
Register number	_____
Submission date	_____

Scope. This submission answers all four questions. Question 1 uses the imported MongoDB restaurant collection; Questions 2-4 present document, wide-column, and graph models designed around their stated access patterns.

Prepared using MongoDB 9.0.4 and Mongosh 2.13.0.

Question 1 - MongoDB Restaurant Queries

The collection contains **25,359** restaurant documents. Large result sets are represented by their exact match count and a compact sample so the output remains readable. Queries can be run unchanged in **mongosh** against **assignment2.restaurants**.

Database	Collection	Documents	Core fields
assignment2	restaurants	25,359	address, borough, cuisine, grades, name, restaurant_id

Q1 Project restaurant ID, name, borough, and zip code; exclude _id

```
db.restaurants.find(
  {},
  { _id: 0, restaurant_id: 1, name: 1,
    borough: 1, "address.zipcode": 1 }
)
```

Output: 25,359 documents. First five are shown below.

Restaurant ID	Name	Borough	Zip code
30075445	Morris Park Bake Shop	Bronx	10462
30112340	Wendy'S	Brooklyn	11225
30191841	Dj Reynolds Pub And Restaurant	Manhattan	10019
40356018	Riviera Caterer	Brooklyn	11224
40356068	Tov Kosher Kitchen	Queens	11374

Q2 Display the first five restaurants in the Bronx

```
db.restaurants.find({ borough: "Bronx" }).limit(5)
```

Output: exactly five complete restaurant documents; identifying fields are shown.

Restaurant ID	Name	Cuisine	Zip code
30075445	Morris Park Bake Shop	Bakery	10462
40357217	Wild Asia	American	10460
40363093	Carvel Ice Cream	Ice Cream, Gelato, Yogurt, Ices	10466
40363289	Happy Garden	Chinese	10474
40364296	Happy Garden	Chinese	10458

Q3 Find restaurants with a score greater than 80 and less than 100

```
db.restaurants.find({
  grades: { $elemMatch: { score: { $gt: 80, $lt: 100 } } }
})
```

Output: **11** restaurants. All matches are listed.

Restaurant ID	Name	Borough	Matching score(s)
40381295	Gandhi	Manhattan	92
40393488	Bella Napoli	Manhattan	98
40756344	West 79Th Street Boat Basin Cafe	Manhattan	89
40979431	Spicy Shallot	Queens	84
40987023	Bistro Caterers	Manhattan	84
41363541	Concrete Restaurant	Manhattan	90
41410058	Anella	Brooklyn	81
41569277	Baluchi'S Indian Food	Manhattan	98
41574642	Cafe R	Manhattan	82
50000040	D & Y Restaurant	Brooklyn	86
50014192	La Potencia Restaurant	Bronx	82

Q4 Find restaurants containing 'Reg' somewhere in the name

```
db.restaurants.find(
  { name: /Reg/ },
  { _id: 0, restaurant_id: 1, name: 1,
    borough: 1, cuisine: 1 }
)
```

Output: **31** matches. First ten are shown.

Restaurant ID	Name	Borough	Cuisine
40356649	Regina Caterers	Brooklyn	American
40369418	Caffe Reggio	Manhattan	Café/Coffee/Tea
40382679	Regency Hotel	Manhattan	American
40402377	Regency Whist Club	Manhattan	American
40523342	Rego Park Cafe	Queens	American
40801325	Regina Pizza	Queens	Pizza
40891782	Regal Entertainment Group	Manhattan	American
41003435	Cafe Regular	Brooklyn	Café/Coffee/Tea
41053746	Rego Pita	Queens	Middle Eastern
41096822	Regional	Manhattan	Italian

Q5 Sort cuisine ascending and borough descending within each cuisine

```
db.restaurants.find({}).sort({ cuisine: 1, borough: -1 })
```

Output: 25,359 sorted documents. First fifteen are shown.

Cuisine	Borough	Restaurant	Restaurant ID
Afghan	Queens	Afghan Kebob House	41535706
Afghan	Queens	Bakhter Afghan Halal Kabab	41559771
Afghan	Queens	Choopan Kabab Restaurant	41569155
Afghan	Queens	Bakhtar Kabab	41661199
Afghan	Queens	New Bkhatar Afghan Halal Kabab & Gyro King	50001906
Afghan	Queens	Balkh Shish Kabab House	50007432
Afghan	Queens	Arya Kabob House	50008452
Afghan	Queens	Tariq Afghan Kabab	50010806
Afghan	Queens	Bakhter Afghan Halal Kabab	50010813
Afghan	Manhattan	Afghan Kebab House	40552806
Afghan	Manhattan	Khyber Pass	40589545
Afghan	Manhattan	Afghan Kebab House #1	40616799
Afghan	Manhattan	Ariana Kebab House	40868400
Afghan	Brooklyn	Bahar Masala	41670224
African	Staten Island	Mona'S Cuisine	41661732

Q6 Find restaurants with at least one grade score below 5

```
db.restaurants.find({ "grades.score": { $lt: 5 } })
```

Output: **8,038** matches. First ten are shown.

Restaurant ID	Name	Borough	Score(s) below 5
30075445	Morris Park Bake Shop	Bronx	2
30191841	Dj Reynolds Pub And Restaurant	Manhattan	2
40356649	Regina Caterers	Brooklyn	2
40357217	Wild Asia	Bronx	4, 3
40357437	C & C Catering Service	Brooklyn	2, 2
40359480	1 East 66Th Street Kitchen	Manhattan	3, 4, 0
40360076	Carvel Ice Cream	Brooklyn	2, 3
40361390	Nordic Delicacies	Brooklyn	4, 3
40361606	The Movable Feast	Brooklyn	2
40361998	Steve Chu'S Deli & Grocery	Queens	3

Q7 Manhattan restaurants having a score of 2 and a score of 6

```
db.restaurants.find({
  borough: "Manhattan",
  "grades.score": { $all: [2, 6] }
})
```

The **$all** operator checks the score values across the embedded grades array. Output: **91** matches; first ten are shown.

Restaurant ID	Name	Borough	Required scores
40364958	White Horse Tavern	Manhattan	2, 6
40365288	Keats Restaurant	Manhattan	2, 6
40367841	Fiorellos	Manhattan	2, 6
40392872	Molly'S	Manhattan	2, 6
40393519	Down The Hatch	Manhattan	2, 6
40393912	Julliard & S.A.B. Cafeteria	Manhattan	2, 6
40397374	Mustang Sally'S Restaurant	Manhattan	2, 6
40398024	Pax Wholesome Foods	Manhattan	2, 6
40399095	Sale & Pepe	Manhattan	2, 6
40400473	Domino'S Pizza	Manhattan	2, 6

Q8 Find the highest score for each restaurant

```
db.restaurants.aggregate([
  { $project: {
      _id: 0, restaurant_id: 1, name: 1,
      highest_score: { $max: "$grades.score" }
  }},
  { $sort: { highest_score: -1, name: 1 } }
])
```

Output: one result per restaurant. The ten highest maximum scores are shown.

Restaurant ID	Restaurant	Highest score
40372466	Murals On 54/Randolphs'S	131
41569277	Baluchi'S Indian Food	98
40393488	Bella Napoli	98
40381295	Gandhi	92
41363541	Concrete Restaurant	90
40756344	West 79Th Street Boat Basin Cafe	89
50000040	D & Y Restaurant	86
40987023	Bistro Caterers	84
40979431	Spicy Shallot	84
41574642	Cafe R	82

Q9 Find names and addresses of restaurants having both A and B grades

```
db.restaurants.find(
  { "grades.grade": { $all: ["A", "B"] } },
  { _id: 0, name: 1, address: 1 }
)
```

Output: **7,832** matches. First ten are shown.

Restaurant	Street address	Zip code
Morris Park Bake Shop	1007 Morris Park Ave	10462
Wendy'S	469 Flatbush Avenue	11225
Tov Kosher Kitchen	97-22 63 Road	11374
May May Kitchen	1269 Sutter Avenue	11208
Seuda Foods	705 Kings Highway	11223
Glorious Food	522 East 74 Street	10021
P & S Deli Grocery	730 Columbus Avenue	10025
Happy Garden	1236 238 Spofford Ave	10474
Tony'S Deli	1069 Wyckoff Avenue	11385
Hot Bagels	7905 Metropolitan Avenue	11379

Q10 Find the Turkish restaurant with the highest average score

```
db.restaurants.aggregate([
  { $match: { cuisine: "Turkish" } },
  { $project: {
      _id: 0, restaurant_id: 1, name: 1,
      borough: 1, cuisine: 1,
      average_score: { $avg: "$grades.score" }
  }},
  { $sort: { average_score: -1 } },
  { $limit: 1 }
])
```

Output: highest average among 70 Turkish restaurants.

Restaurant ID	Name	Borough	Average score
50017871	Cocoa Grinder	Brooklyn	33

Question 2 - Document Data Model

A course is the aggregate root. Modules and resources are embedded because they are normally read and updated in the context of one course. Instructor details are included as a snapshot; an instructor ID can also reference a separate instructor collection when profiles are shared across many courses.

```
{
  "_id": ObjectId("..."),
  "course_code": "DS301",
  "title": "Applied Data Science with Python",
  "description": "Data preparation, modelling and evaluation.",
  "category": "Data Science",
  "level": "Intermediate",
  "instructor": {
    "instructor_id": "INS104",
    "name": "Dr. Maya Rao",
    "email": "maya.rao@example.edu",
    "expertise": ["Python", "Machine Learning"]
  },
  "prerequisites": [
    { "course_code": "PY101", "title": "Python Fundamentals" },
    { "course_code": "ST101", "title": "Basic Statistics" }
  ],
  "modules": [
    {
      "module_no": 1,
      "title": "Data Preparation",
      "sequence": 1,
      "estimated_minutes": 180,
      "resources": [
        { "resource_id": "R101", "type": "video",
          "title": "Cleaning Data", "url": "/videos/r101",
          "duration_minutes": 24 },
        { "resource_id": "R102", "type": "reading",
          "title": "Missing Values", "url": "/readings/r102",
          "estimated_minutes": 20 },
        { "resource_id": "R103", "type": "quiz",
          "title": "Preparation Quiz", "questions": 10,
          "max_score": 20, "attempts_allowed": 2 },
        { "resource_id": "R104", "type": "assignment",
          "title": "Clean a Dataset", "max_score": 50,
          "submission_type": "notebook" }
      ]
    }
  ],
  "metadata": {
    "language": "English", "duration_hours": 24,
    "tags": ["python", "analytics", "machine-learning"],
    "published": true, "version": 3
  },
  "created_at": ISODate("2026-08-01T09:00:00Z"),
  "updated_at": ISODate("2026-10-01T11:30:00Z")
}
```

Design Decisions and Indexes

Decision	Reason
Embed modules	Modules belong to one course and are normally retrieved with it.
Embed resources	Different resource types can use flexible type-specific fields.
Instructor snapshot	Preserves the displayed instructor data; instructor_id supports a separate master record.
Prerequisite references	Course codes avoid duplicating complete prerequisite course documents.
Version and timestamps	Support publishing workflow, cache invalidation and auditability.

```
db.courses.createIndex({ course_code: 1 }, { unique: true })
db.courses.createIndex({ category: 1, level: 1 })
db.courses.createIndex({ "instructor.instructor_id": 1 })
db.courses.createIndex({ "metadata.tags": 1 })
db.courses.createIndex({ "modules.resources.type": 1 })
```

Scaling note. If a course grows close to MongoDB's document-size limit or resources need frequent independent updates, store resource metadata in a separate **resources** collection and keep only resource IDs and ordering information inside each module.

Question 3 - Wide-Column Data Model

The main access pattern is: retrieve one passenger's newest transactions without scanning other passengers. A month bucket keeps partitions bounded, while transaction_timestamp in descending order places the newest transaction first.

Primary table: passenger history by month

```
CREATE TABLE passenger_transactions_by_month (
  passenger_id        text,
  history_month       text,       // Example: "2026-10"
  transaction_ts      timestamp,
  booking_reference   text,
  train_number        text,
  journey_date        date,
  transaction_type    text,       // BOOKING, CANCELLATION, MODIFY
  transaction_status  text,       // SUCCESS, FAILED, PENDING
  from_station        text,
  to_station          text,
  travel_class        text,
  amount              decimal,
  PRIMARY KEY (
    (passenger_id, history_month),
    transaction_ts,
    booking_reference
  )
) WITH CLUSTERING ORDER BY (
  transaction_ts DESC,
  booking_reference ASC
);
```

Key component	Columns	Purpose
Partition key	passenger_id, history_month	Routes one passenger-month to one partition and prevents unbounded growth.
Clustering keys	transaction_ts DESC, booking_reference	Returns newest records first and uniquely orders same-time events.
Regular columns	train, journey, type, status, route, class, amount	Makes each transaction row self-contained for fast reads.

Query for recent passenger transactions

```
SELECT *
FROM passenger_transactions_by_month
WHERE passenger_id = 'P10027'
  AND history_month = '2026-10'
LIMIT 20;
```

For history spanning several months, the application queries the required month buckets in parallel and merges the already time-ordered results.

Secondary table: audit trail by booking reference

Wide-column systems commonly duplicate data for a second access pattern. This table retrieves every booking, modification and cancellation for a booking reference without filtering the passenger table.

```
CREATE TABLE booking_transactions_by_reference (
  booking_reference   text,
  transaction_ts      timestamp,
  passenger_id        text,
  train_number        text,
  journey_date        date,
  transaction_type    text,
  transaction_status  text,
  PRIMARY KEY (booking_reference, transaction_ts)
) WITH CLUSTERING ORDER BY (transaction_ts DESC);
```

```
SELECT *
FROM booking_transactions_by_reference
WHERE booking_reference = 'BR893541';
```

Passenger ID	Month	Timestamp	Booking ref	Train	Type	Status
P10027	2026-10	2026-10-19 09:42	BR893541	12628	MODIFY	SUCCESS
P10027	2026-10	2026-10-18 20:11	BR893541	12628	BOOKING	SUCCESS

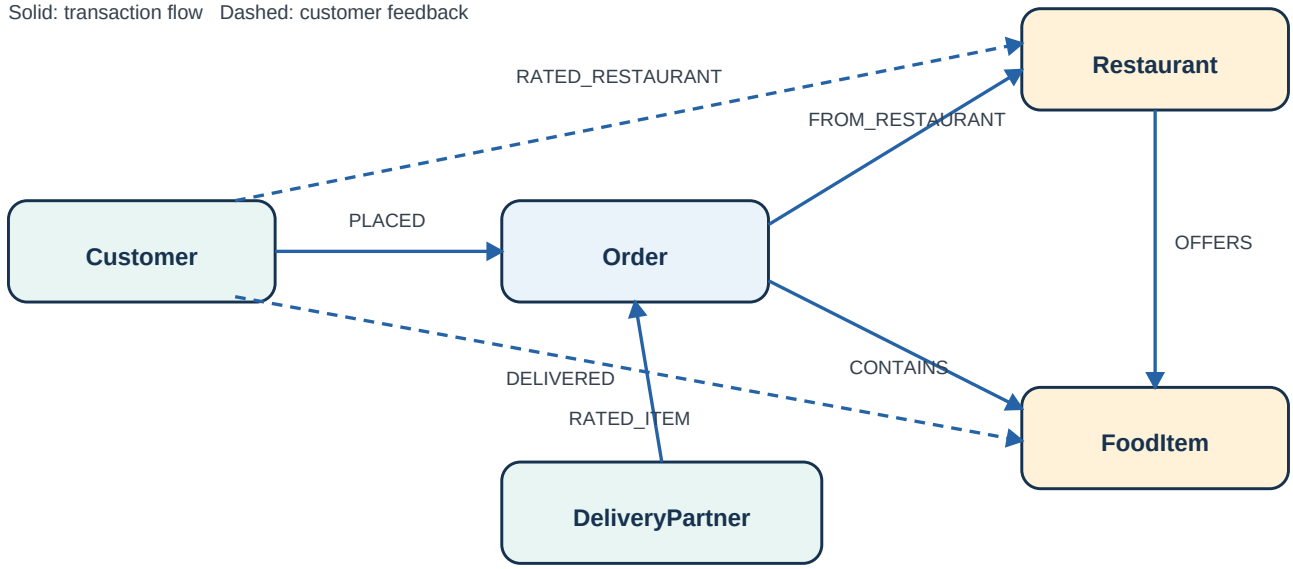
Write strategy. Each transaction is written to both tables. This controlled denormalization is intentional: it trades a small amount of storage for predictable, scan-free reads.

Question 4 - Graph Data Model

Orders are represented as nodes because they carry status, time and amount and connect several actors. Ratings are relationships because they express a customer's opinion about a restaurant or food item. This structure supports order history, fulfilment tracking and recommendations through short graph traversals.

Core property graph

Solid: transaction flow Dashed: customer feedback



Node Types and Properties

Node label	Key properties	Purpose
Customer	customer_id, name, city	Person placing orders and creating ratings.
Restaurant	restaurant_id, name, cuisine, location, avg_rating	Merchant from which an order is placed.
FoodItem	item_id, name, category, price, available	Menu item offered by a restaurant.
Order	order_id, ordered_at, total_amount, status	Transaction connecting customer, restaurant, items and delivery.
DeliveryPartner	partner_id, name, vehicle_type, rating	Partner assigned to fulfil an order.

Relationship Types and Properties

Relationship	Direction	Relevant properties
PLACED	Customer -> Order	ordered_at
FROM_RESTAURANT	Order -> Restaurant	restaurant snapshot or branch_id (optional)
CONTAINS	Order -> FoodItem	quantity, unit_price, customization
OFFERS	Restaurant -> FoodItem	current_price, available_since
DELIVERED	DeliveryPartner -> Order	assigned_at, picked_up_at, delivered_at
RATED_RESTAURANT	Customer -> Restaurant	rating, comment, reviewed_at
RATED_ITEM	Customer -> FoodItem	rating, comment, reviewed_at
SIMILAR_TO	Restaurant <-> Restaurant	similarity_score, basis, calculated_at

Recommendation traversal

```
MATCH (c:Customer {customer_id: $customerId})
      -[:PLACED]->(:Order)-[:FROM_RESTAURANT]->(visited:Restaurant)
MATCH (visited)-[s:SIMILAR_TO]-(candidate:Restaurant)
WHERE NOT EXISTS {
  MATCH (c)-[:PLACED]->(:Order)
        -[:FROM_RESTAURANT]->(candidate)
}
RETURN candidate.restaurant_id AS restaurant_id,
       candidate.name AS restaurant,
       candidate.cuisine AS cuisine,
       max(s.similarity_score) AS score
ORDER BY score DESC, candidate.avg_rating DESC
LIMIT 10;
```

The traversal starts with restaurants from the customer's previous orders, follows precomputed similarity edges, removes restaurants already ordered from, and ranks the remaining candidates. Similarity can combine shared cuisine, overlapping menu categories, geographic distance and rating patterns.

Conclusion

The four solutions demonstrate query-driven NoSQL modelling. MongoDB embeds inspection grades within each restaurant, the learning platform embeds course-owned modules and polymorphic resources, the railway model uses partition and clustering keys for bounded chronological reads, and the food-delivery graph makes customer-order-restaurant relationships directly traversable.

Question	Model / technique	Primary design objective
Q1	MongoDB queries and aggregation	Correct filtering, projection, sorting and array analysis.
Q2	Document model	Flexible course structures retrieved as cohesive aggregates.
Q3	Wide-column model	Fast passenger history reads with bounded partitions.
Q4	Property graph	Efficient order-history traversal and restaurant recommendations.

Reproducibility. Query counts and displayed examples were generated from the locally imported MongoDB primer restaurant dataset containing 25,359 newline-delimited JSON documents.